

Eigen commando's (Agenda)

We gaan kijken hoe je je eigen commando's kunt aanmaken. We gaan kijken naar:

Eigen commando's (Agenda)

We gaan kijken hoe je je eigen commando's kunt aanmaken. We gaan kijken naar:

- Nieuwe commando's

Eigen commando's (Agenda)

We gaan kijken hoe je je eigen commando's kunt aanmaken. We gaan kijken naar:

- Nieuwe commando's
- Nieuwe commando's met één of meerdere *inputs*

Eigen commando's (Agenda)

We gaan kijken hoe je je eigen commando's kunt aanmaken. We gaan kijken naar:

- Nieuwe commando's
- Nieuwe commando's met één of meerdere *inputs*
- Herdefiniëren van commando's

Eigen commando's (Agenda)

We gaan kijken hoe je je eigen commando's kunt aanmaken. We gaan kijken naar:

- Nieuwe commando's
- Nieuwe commando's met één of meerdere *inputs*
- Herdefiniëren van commando's
- Operatoren

Eigen commando's (Agenda)

We gaan kijken hoe je je eigen commando's kunt aanmaken. We gaan kijken naar:

- Nieuwe commando's
- Nieuwe commando's met één of meerdere *inputs*
- Herdefiniëren van commando's
- Operatoren
- Oefeningetje

Eigen Commando's

Je kunt je **eigen commando's** aanmaken om veel typwerk te voorkomen. Je doet dit als volgt:

1

```
\newcommand{\naam}{wat je wil dat je commando doet}
```

Eigen Commando's

Je kunt je **eigen commando's** aanmaken om veel typwerk te voorkomen. Je doet dit als volgt:

```
1 \newcommand{\naam}{wat je wil dat je commando doet}
```

Je zet `\newcommand` meestal voor je `\begin{document}`:

```
1 \documentclass[a4paper]{article}
2 \newcommand{\mbbR}{\mathbb{R}}
3 \begin{document}
4 We kunnen nu ons nieuwe commando aanroepen:  $x \in \mathbb{R}$ .
5 Dit is sneller dan  $x \in \mathbb{R}$ 
6 \end{document}
```

Eigen Commando's

Je kunt je **eigen commando's** aanmaken om veel typwerk te voorkomen. Je doet dit als volgt:

```
1 \newcommand{\naam}{wat je wil dat je commando doet}
```

Je zet `\newcommand` meestal voor je `\begin{document}`:

```
1 \documentclass[a4paper]{article}
```

```
2 \newcommand{\mbbR}{\mathbb{R}}
```

```
3 \begin{document}
```

```
4 We kunnen nu ons nieuwe commando aanroepen: $x \in \mbbR$.
```

```
5 Dit is sneller dan $x \in \mathbb{R}$
```

```
6 \end{document}
```

We kunnen nu ons nieuwe commando aanroepen: $x \in \mathbb{R}$. Dit is sneller dan $x \in \mathbb{R}$

Eigen Commando's

Je kunt ook je eigen commando's maken die nog een **input** nodig hebben. Je doet dit met `\newcommand{\naam}[1]{wat je wil dat het doet}`. Bijvoorbeeld

1

```
\newcommand{\bb}[1]{\mathbb{#1}}
```

Eigen Commando's

Je kunt ook je eigen commando's maken die nog een **input** nodig hebben. Je doet dit met `\newcommand{\naam}[1]{wat je wil dat het doet}`. Bijvoorbeeld

1

```
\newcommand{\bb}[1]{\mathbb{#1}}
```

Let op dat je hier één argument *moet* invoeren!

Eigen Commando's

Je kunt ook je eigen commando's maken die nog een **input** nodig hebben. Je doet dit met `\newcommand{\naam}[1]{wat je wil dat het doet}`. Bijvoorbeeld

```
1 \newcommand{\bb}[1]{\mathbb{#1}}
```

Let op dat je hier één argument *moet* invoeren! Dus bijvoorbeeld:

```
1 \documentclass[a4paper]{article}
2 \newcommand{\bb}[1]{\mathbb{#1}}
3 \begin{document}
4 De reële getallen zijn  $\bb{R}$ , de breuken zijn  $\bb{Q}$ .
5 \end{document}
```

Eigen Commando's

Je kunt ook je eigen commando's maken die nog een **input** nodig hebben. Je doet dit met `\newcommand{\naam}[1]{wat je wil dat het doet}`. Bijvoorbeeld

```
1 \newcommand{\bb}[1]{\mathbb{#1}}
```

Let op dat je hier één argument *moet* invoeren! Dus bijvoorbeeld:

```
1 \documentclass[a4paper]{article}
2 \newcommand{\bb}[1]{\mathbb{#1}}
3 \begin{document}
4 De reële getallen zijn  $\mathbb{R}$ , de breuken zijn  $\mathbb{Q}$ .
5 \end{document}
```

De reële getallen zijn $\mathbb{R}$, de breuken zijn $\mathbb{Q}$.

Eigen Commando's

Voor een commando met **meerdere argumenten** veranderen we de [1] naar bijvoorbeeld [2]:

```
1 \documentclass[a4paper]{article}
2 \newcommand{\der}[2]{\frac{d #1}{d #2}}
3 \begin{document}
4 De afgeleide van  $f$  is  $\frac{d f}{d x}$ 
5 \end{document}
```

Eigen Commando's

Voor een commando met **meerdere argumenten** veranderen we de [1] naar bijvoorbeeld [2]:

```
1 \documentclass[a4paper]{article}
2 \newcommand{\der}[2]{\frac{d #1}{d #2}}
3 \begin{document}
4 De afgeleide van  $f$  is  $\frac{d}{dx} f$ 
5 \end{document}
```

Dit geeft:

De afgeleide van f is $\frac{df}{dx}$

Eigen Commando's

Soms is een commando al gedefinieerd. Je kunt het commando dan 'herschrijven', mits het niet al te belangrijk is. Een voorbeeld is `\P`. Als je probeert om dit te definiëren:

```
1 \newcommand{\P}{\mathbb{P}}
```

dan krijg je een foutmelding

Command `\P` already defined

Eigen Commando's

Soms is een commando al gedefinieerd. Je kunt het commando dan 'herschrijven', mits het niet al te belangrijk is. Een voorbeeld is `\P`. Als je probeert om dit te definiëren:

```
1 \newcommand{\P}{\mathbb{P}}
```

dan krijg je een foutmelding

Command `\P` already defined

Om het commando te **herschrijven**:

```
1 \renewcommand{\P}{\mathbb{P}}
```

Eigen Commando's

Soms is een commando al gedefinieerd. Je kunt het commando dan 'herschrijven', mits het niet al te belangrijk is. Een voorbeeld is `\P`. Als je probeert om dit te definiëren:

```
1 \newcommand{\P}{\mathbb{P}}
```

dan krijg je een foutmelding

Command `\P` already defined

Om het commando te **herschrijven**:

```
1 \renewcommand{\P}{\mathbb{P}}
```

Een veelvoorkomend voorbeeld is `\renewcommand{\vec}{\mathbf}` om $\vec{v}$ in plaats van $\vec{v}$ te krijgen.

Eigen Commando's (operatoren)

Sommige functies, zoals $\sin(\cdot)$ zijn – in tegenstelling tot normale letters in mathmode – rechtgedrukt. Als je ook een commando wil maken voor iets wat rechtgedrukt moet staan, dan kan je dat doen met de `\operatorname{}` command.

Eigen Commando's (operatoren)

Sommige functies, zoals $\sin(\cdot)$ zijn – in tegenstelling tot normale letters in mathmode – rechtgedrukt. Als je ook een commando wil maken voor iets wat rechtgedrukt moet staan, dan kan je dat doen met de `\operatorname{}` command.

```
1 \documentclass[a4paper]{article}
2 \newcommand{\Li}{\operatorname{Li}}
3 \begin{document}
4 De `logarithmic integral function'  $\Li$  is definieerd als
5  $\displaystyle \Li(x) = \int_0^1 \frac{dt}{\log(t)}$ .
6 \end{document}
```

Eigen Commando's (operatoren)

Sommige functies, zoals $\sin(\cdot)$ zijn – in tegenstelling tot normale letters in mathmode – rechtgedrukt. Als je ook een commando wil maken voor iets wat rechtgedrukt moet staan, dan kan je dat doen met de `\operatorname{}` command.

```

1 \documentclass[a4paper]{article}
2 \newcommand{\Li}{\operatorname{Li}}
3 \begin{document}
4 De 'logarithmic integral function'  $\Li$  is definieerd als
5  $\displaystyle \Li(x) = \int_0^1 \frac{dt}{\log(t)}.$ 
6 \end{document}

```

De 'logarithmic integral function' Li is definieerd als $\operatorname{Li}(x) := \int_0^1 \frac{dt}{\log(t)}.$

Oefening

Maak een commando zodat `\gem{x}{w}` het volgende weergeeft:

$$\text{Gem}^y(x)$$

Let op, Gem is *rechtgedrukt*!

Oefening

Maak een commando zodat `\gem{x}{w}` het volgende weergeeft:

$$\text{Gem}^y(x)$$

Let op, Gem is *rechtgedrukt*!

Antwoord

```
\newcommand{\gem}[1]{\operatorname{Gem}^{\#2}(\#1)}
```

Comments (Agenda)

We gaan **comments** bespreken. Een comment is een stukje code wat genegeerd zal worden bij het uitvoeren van je code. Dit kan soms erg handig zijn.

We gaan het volgende bespreken:

Comments (Agenda)

We gaan **comments** bespreken. Een comment is een stukje code wat genegeerd zal worden bij het uitvoeren van je code. Dit kan soms erg handig zijn.

We gaan het volgende bespreken:

- Hoe maak je comments

Comments (Agenda)

We gaan **comments** bespreken. Een comment is een stukje code wat genegeerd zal worden bij het uitvoeren van je code. Dit kan soms erg handig zijn.

We gaan het volgende bespreken:

- Hoe maak je comments
- Waarvoor is dit handig?

Comments (Hoe Maak Je Comments?)

In $\text{\LaTeX}$ kun je – net als in andere programmeertalen – comments maken binnen je code. Je geeft aan wat een comment moet zijn door een %-teken aan het begin van de regel te zetten. Je kunt in Overleaf ook `Ctrl + ?` typen. Dan komt er een % aan het begin van de regel.

$\text{\LaTeX}$ negeert *alles* na het %-teken.

Comments (Hoe Maak Je Comments?)

In $\text{\LaTeX}$ kun je – net als in andere programmeertalen – comments maken binnen je code. Je geeft aan wat een comment moet zijn door een %-teken aan het begin van de regel te zetten. Je kunt in Overleaf ook `Ctrl + ?` typen. Dan komt er een % aan het begin van de regel.

$\text{\LaTeX}$ negeert *alles* na het %-teken. Voorbeeld:

```
1 \documentclass[a4paper]{article}
2 \begin{document}
3 Hier is tekst die wel weergegeven wordt.
4 % Deze tekst wordt niet weergegeven.
5 \end{document}
```

Comments (Hoe Maak Je Comments?)

In $\text{\LaTeX}$ kun je – net als in andere programmeertalen – comments maken binnen je code. Je geeft aan wat een comment moet zijn door een %-teken aan het begin van de regel te zetten. Je kunt in Overleaf ook `Ctrl + ?` typen. Dan komt er een % aan het begin van de regel.

$\text{\LaTeX}$ negeert *alles* na het %-teken. Voorbeeld:

```
1 \documentclass[a4paper]{article}
2 \begin{document}
3 Hier is tekst die wel weergegeven wordt.
4 % Deze tekst wordt niet weergegeven.
5 \end{document}
```

Dit geeft dan:

Hier is tekst die wel weergegeven wordt.

Comments (Waarvoor?)

Comments zijn nuttig voor twee belangrijke redenen:

Comments (Waarvoor?)

Comments zijn nuttig voor twee belangrijke redenen:

- Tekst **opslaan**;

Comments (Waarvoor?)

Comments zijn nuttig voor twee belangrijke redenen:

- Tekst **opslaan**;
 - ▶ Als je een stuk van een opdracht hebt uitgetypt, en je komt erachter dat het niet helemaal klopt, dan is het vaak handig om dit niet allemaal te verwijderen, maar het gewoon in een comment te zetten. Zo blijft het toch opgeslagen in je bestand, zonder dat het weergegeven wordt.

Comments (Waarvoor?)

Comments zijn nuttig voor twee belangrijke redenen:

- Tekst **opslaan**;
 - ▶ Als je een stuk van een opdracht hebt uitgetypt, en je komt erachter dat het niet helemaal klopt, dan is het vaak handig om dit niet allemaal te verwijderen, maar het gewoon in een comment te zetten. Zo blijft het toch opgeslagen in je bestand, zonder dat het weergegeven wordt.
- Om **foutmeldingen** te verhelpen;

Comments (Waarvoor?)

Comments zijn nuttig voor twee belangrijke redenen:

- Tekst **opslaan**;
 - ▶ Als je een stuk van een opdracht hebt uitgetypt, en je komt erachter dat het niet helemaal klopt, dan is het vaak handig om dit niet allemaal te verwijderen, maar het gewoon in een comment te zetten. Zo blijft het toch opgeslagen in je bestand, zonder dat het weergegeven wordt.
- Om **foutmeldingen** te verhelpen;
 - ▶ Bij een foutmelding die je niet snapt →comment je code *helemaal* en compile je document. Als je nu geen errors krijgt, dan heb je het stuk met de fout te pakken. Je kunt nu langzaam regels gaan uncommenten en je bestand weer compilen om zo te vinden waar de error zit.

Packages (Inleiding)

We gaan een aantal packages bespreken die handig zijn om te kennen en te gebruiken. Een paar hiervan zijn al besproken. De overigen zullen we bespreken.

- Handige packages
- Korte bespreking van enumitem
- Korte bespreking van hyperref en cleveref
- Probeer het zelf eens!

Packages

Packages staan in de **preamble** van een document om meer commando's in het document te kunnen gebruiken. Handige packages zijn onderanderen:

- Voor wiskunde: `amssymb`, `amsmath`, `amsthm`, `mathrsfs`
- Voor taal: `babel`, `inputenc`, `fontenc`
- Voor afbeeldingen: `graphicx`, `float`, `tikz`
- Voor lijsten: `enumitem`
- Voor linkjes: `hyperref`, `cleveref`
- Voor layout: `geometry`, `fancyhdr`

Enumitem

Een genummerde lijst kan gemaakt worden met

```

1 \begin{enumerate}[label=\arabic*]
2     \item pen
3     \item papier
4     \item laptop
5 \end{enumerate}

```

en een ongenummerde lijst met

```

6 \begin{itemize}[label=\textbullet]
7     \item lunch
8     \item waterfles
9 \end{itemize}

```

worden met

- 1 pen
- 2 papier
- 3 laptop

en een ongenummerde lijst met

- lunch
- waterfles

Hyperref en cleveref

1 Stel dat ik wil terug refereren naar

2 `\begin{align}\label{eq:cirkel}`

3 $x^2 + y^2 = a$

4 `\end{align}`

5 dan kan ik nu gemakkelijk terug

6 refereren naar een formule

7 van de cirkel met `\cref{eq:cirkel}`.

8 In plaats hiervan kan je ook `\ref{eq:cirkel}` gebruiken.

9 Dit kan je ook doen voor figuren, sections en meer.

Stel dat ik wil terug refereren naar

$$x^2 + y^2 = a \quad (1)$$

dan kan ik nu gemakkelijk terug refereren
naar een formule van de cirkel met

`eq:cirkel2`. In plaats hiervan kan je ook

`??` gebruiken. Dit kan je ook doen voor
figuren, sections en meer.

Probeer het zelf

Probeer zelf eens:

- i een genummerde lijst te maken,
- ii een ongenummerde lijst te maken,
- iii een label te geven aan een section, figuur, stelling of iets anders,
- iv als je een label hebt, probeer er nu naar te refereren.

Tekstopmaak

Resultaat	Code	Resultaat	Code
Tekst	<code>\textbf{Tekst}</code>	Tekst	<code>\tiny Tekst</code>
<i>Tekst</i>	<code>\textit{Tekst}</code>	Tekst	<code>\small Tekst</code>
TEKST	<code>\textsc{Tekst}</code>	Tekst	<code>\large Tekst</code>
<u>Tekst</u>	<code>\underline{Tekst}</code>	Tekst	<code>\huge Tekst</code>
Tekst	<code>\mathbf{Tekst}</code>		
$\mathbb{T}$	<code>\mathbb{T}</code>		
$\mathcal{T}$	<code>\mathcal{T}</code>		
$\mathfrak{T}$	<code>\mathfrak{T}</code>		
α	<code>\boldsymbol{\alpha}</code>		

Zelf oplossingen vinden

$\text{\LaTeX}$ biedt veel opties, wat zowel handig als frustrerend kan zijn. Daarom is het belangrijk om te leren zoeken naar wat je wil bereiken in je document.

Het helpt om **duidelijke zoekopdrachten** te hebben, zoals bijvoorbeeld 'tabellen maken latex', 'citing in latex', 'error ... latex'.

Handige websites om te bezoeken:

`https://tex.stackexchange.com/`

`https://www.overleaf.com/learn/latex/Learn\_LaTeX\_in\_30\_minutes`

`https://nl.wikibooks.org/wiki/LaTeX/Wiskundige\_formules`

`https://detexify.kirelabs.org/classify.html`

Afsluiting

...